# Backend'e Giriş

## Node.js + Express — Part 1

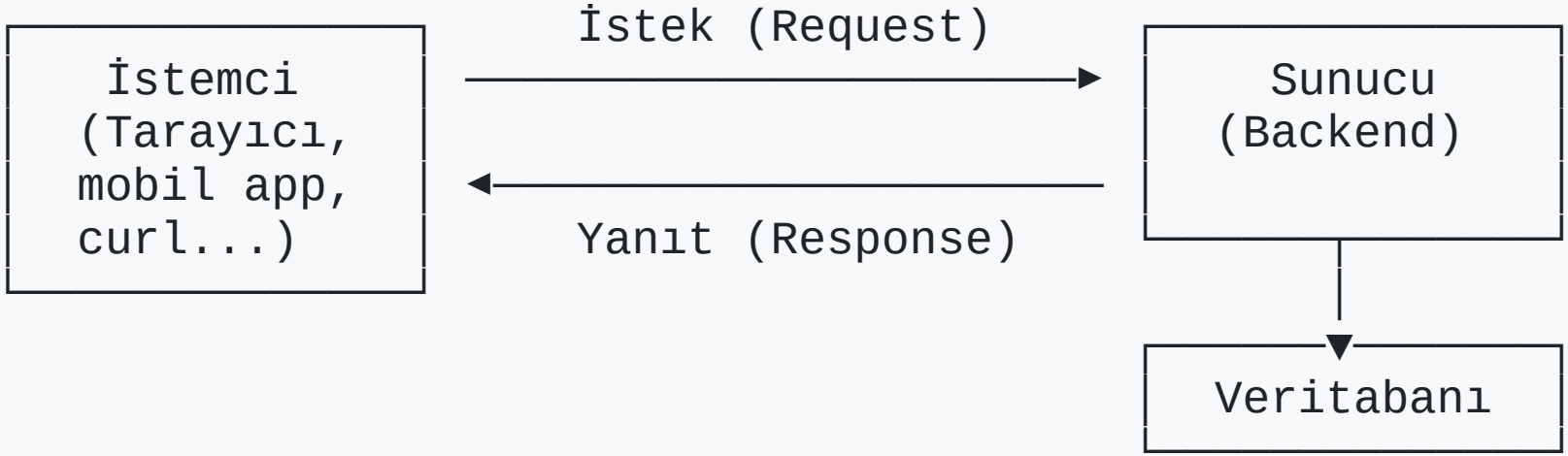
HTTP'den ilk REST API'mize

# Bugünkü yol haritası

1. Web ve HTTP nasıl çalışır?
2. Node.js nedir, neden farklı?
3. Framework'süz bir sunucu
4. Express ile tanışma
5. Middleware
6. **Canlı kodlama:** Todo API + JSON dosyasıyla kalıcılık
7. Sırada ne var? (Part 2 kavramları)

# 1. Web ve HTTP

# İstemci ve Sunucu




- **Frontend:** kullanıcının gördüğü kısım
- **Backend:** veriyi saklayan, iş kurallarını uygulayan, güvenliği sağlayan kısım

# Bir HTTP isteğinin anatomisi

```
POST /api/todos HTTP/1.1            ← Metot + Yol (path)
Host: example.com                   ← Header'lar
Content-Type: application/json
Authorization: Bearer abc123

{ "title": "Süt al" }               ← Body (gövde)
```

- **Metot:** ne yapmak istiyoruz?
- **Yol (URL):** hangi kaynak üzerinde?
- **Header'lar:** ek bilgiler (format, kimlik, dil...)
- **Body:** gönderilen veri (GET'te genelde yok)

# Bir HTTP yanıtının anatomisi

```
HTTP/1.1 201 Created                ← Durum kodu
Content-Type: application/json      ← Header'lar

{ "id": 3, "title": "Süt al", "done": false }   ← Body
```

HTTP **durumsuzdur (stateless):** her istek kendi başına bir bütündür, sunucu bir önceki isteği "hatırlamaz".

# HTTP metotları

| Metot | Anlamı | Örnek |
|---|---|---|
| `GET` | Oku | `GET /todos` |
| `POST` | Oluştur | `POST /todos` |
| `PUT` | Tamamen değiştir | `PUT /todos/3` |
| `PATCH` | Kısmen güncelle | `PATCH /todos/3` |
| `DELETE` | Sil | `DELETE /todos/3` |

`GET` hiçbir şeyi değiştirmemeli. Aynı `PUT` / `DELETE` isteğini 2 kez atmak, 1 kez atmakla aynı sonucu vermeli (**idempotent**).

# Durum kodları

| Aralık | Anlamı | Sık kullanılanlar |
|---|---|---|
| **2xx** | Başarılı | `200 OK`, `201 Created`, `204 No Content` |
| **3xx** | Yönlendirme | `301`, `304 Not Modified` |
| **4xx** | **İstemcinin** hatası | `400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found` |
| **5xx** | **Sunucunun** hatası | `500 Internal Server Error`, `503` |

> Kural: hata kimdeyse kodu ona göre seç. Boş form → 400. Bizim kodumuz patladı → 500.

# JSON: backend'in ortak dili

```
{
  "id": 1,
  "title": "Node.js kur",
  "done": true,
  "tags": ["kurulum", "araçlar"]
}
```

- Metin tabanlı, okunabilir, her dilde parser'ı var
- JavaScript nesnesine çok benziyor (ama anahtarlar **çift tırnaklı**)

**Demo:** `curl -i https://api.github.com/users/octocat`

# 2. Node.js

# Node.js nedir?

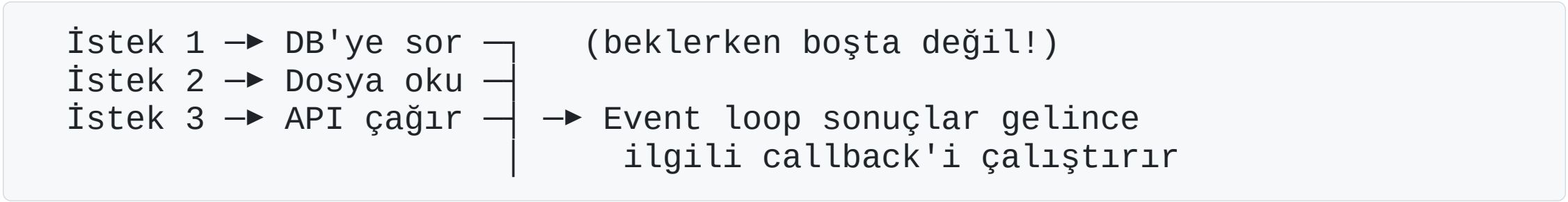
- Tarayıcı dışında JavaScript çalıştıran bir **runtime**
- Chrome'un **V8** motoru + işletim sistemi erişimi (dosya, ağ, süreç)
- **npm:** dünyanın en büyük paket deposu (Python'daki pip, Java'daki Maven gibi)

```
node --version          # 22+ olmalı
npm init -y             # package.json oluştur
npm install express     # paket kur → node_modules/
```

# Tek thread, ama bloklamayan I/O

Java/PHP klasik model: **her istek bir thread**

Node modeli: **tek thread + event loop**




- Backend'in zamanının çoğu **beklemekle** geçer (DB, ağ, disk)
- Node beklerken başka isteklerle ilgilenir
- ⚠️ Ağır CPU işi (ör. büyük hesaplama) thread'i **bloklar**

# async / await

```
const bekle = (ms) => new Promise((r) => setTimeout(r, ms));

async function veriGetir(isim, ms) {
  await bekle(ms);            // ör. veritabanı sorgusu
  return `${isim} hazır`;
}

// Sıralı: ~1000 ms
await veriGetir('Kullanıcı', 500);
await veriGetir('Siparişler', 500);

// Paralel: ~500 ms
await Promise.all([veriGetir('Kullanıcı', 500), veriGetir('Siparişler', 500)]);
```

Python `asyncio` veya C# `async/await` ile aynı fikir.

📁 `npm run ornek:1`

# 3. Framework'süz sunucu

# `node:http` ile sunucu

```
import http from 'node:http';

const server = http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end('Merhaba!');
    return;
  }
  res.writeHead(404);
  res.end('Bulunamadı');
});

server.listen(3000);
```

📂 `npm run ornek:2`

# Peki neden framework?

Elle yaptığımızda:

- `/kullanicilar/42` gibi **dinamik URL'leri** kendimiz parse etmeliyiz
- **Body'yi** stream olarak parça parça okumalıyız
- `?sayfa=2` gibi **query string'leri** ayrıştırmalıyız
- Log, yetki kontrolü gibi işleri **her route'ta tekrar** yazmalıyız

➡️ **Express** bunları çözen minimal bir katman
(Python'daki **Flask**, Ruby'deki **Sinatra** benzeri)

# 4. Express

# İlk Express uygulaması

```
import express from 'express';

const app = express();

app.get('/', (req, res) => {
  res.send('Merhaba Express!');
});

app.get('/api/saat', (req, res) => {
  res.json({ saat: new Date().toISOString() });
});

app.listen(3000, () => console.log('http://localhost:3000'));
```

`app.METOT(yol, handler)` — her route bir metot + yol + fonksiyon.

# `req` ve `res`

| `req` (gelen istek) | `res` (gidecek yanıt) |
|---|---|
| `req.params` — URL parametreleri | `res.status(201)` — durum kodu |
| `req.query` — query string | `res.json({...})` — JSON gönder |
| `req.body` — gövde (middleware gerekir) | `res.send('...')` — metin gönder |
| `req.get('Header')` — header oku | `res.set('Header', 'x')` — header yaz |
| `req.method`, `req.path` | `res.end()` — gövdesiz bitir |

# `req`: istek nereye düşer?

```
curl -X POST "localhost:3000/api/kullanicilar/42/notlar?sirala=yeni" \
  -H "Content-Type: application/json" -H "X-Dil: tr" \
  -d '{"metin": "Merhaba"}'
```

```
app.use(express.json()); // body'yi okuyabilmek için şart

app.post('/api/kullanicilar/:id/notlar', (req, res) => {
  req.method;         // 'POST'
  req.path;           // '/api/kullanicilar/42/notlar'
  req.params.id;      // '42'        ← string! Number(...) ile çevir
  req.query.sirala;   // 'yeni'      ← o da string
  req.get('X-Dil');   // 'tr'        ← header adları büyük/küçük harf duyarsız
  req.body.metin;     // 'Merhaba'   ← express.json() yoksa undefined
});
```

# `res`: yanıt göndermenin yolları

```
res.json({ id: 7 });                    // 200 + JSON (en sık)
res.status(201).json({ id: 7 });        // durum kodu + JSON (zincirleme)
res.status(204).end();                  // başarılı, gövde yok
res.sendStatus(404);                    // sadece kod: "Not Found"
res.set('X-Toplam', '42').json([]);     // header + JSON
res.send('<h1>Merhaba</h1>');           // metin / HTML
res.redirect('/yeni-adres');            // 302 yönlendirme
```

- `status()` ve `set()` yanıtı **göndermez**, sadece hazırlar → zincirlenebilir
- `json()`, `send()`, `end()`, `redirect()` yanıtı **gönderir ve bitirir**
- Her istek için **sadece bir kez** yanıt gönderilir

# Klasik hatalar

```
app.get('/todos/:id', (req, res) => {
  const todo = todos.find((t) => t.id === req.params.id); // ❌ '1' === 1 → false
  if (!todo) {
    res.status(404).json({ error: 'Yok' });              // ❌ return yok!
  }
  res.json(todo);  // 💥 Cannot set headers after they are sent
});
```

```
app.get('/todos/:id', (req, res) => {
  const todo = todos.find((t) => t.id === Number(req.params.id)); // ✅
  if (!todo) {
    return res.status(404).json({ error: 'Yok' });               // ✅
  }
  res.json(todo);
});
```

Bir de: hiç yanıt göndermezsen istemci **sonsuza kadar bekler** (timeout).

# Params ve query

```
// GET /api/kullanicilar?sehir=İstanbul
app.get('/api/kullanicilar', (req, res) => {
  const { sehir } = req.query;
  res.json(sehir ? kullanicilar.filter((k) => k.sehir === sehir) : kullanicilar);
});

// GET /api/kullanicilar/2
app.get('/api/kullanicilar/:id', (req, res) => {
  const id = Number(req.params.id);   // ⚠ params her zaman string!
  const kullanici = kullanicilar.find((k) => k.id === id);
  if (!kullanici) {
    return res.status(404).json({ hata: 'Kullanıcı bulunamadı' });
  }
  res.json(kullanici);
});
```

📁 `npm run ornek:3`

# 5. Middleware

# Middleware nedir?

İstek ile route arasında çalışan **ara fonksiyonlar zinciri**:

```
İstek ─► [logger] ─► [express.json] ─► [yetki] ─► [route] ─► Yanıt
                                                    │ hata
                                                    ▼
                                              [hata middleware'i]
```

Benzetme: **havalimanı kontrolleri —** bilet kontrolü → güvenlik → pasaport → uçak. Her nokta yolcuyu bir sonrakine geçirebilir, geri çevirebilir ya da üzerine bir şey ekleyebilir (damga, etiket).

Diğer dillerden: Django/Flask *middleware*, Java *Servlet Filter*, ASP.NET *pipeline*.

# Bir middleware'in imzası

```
function logger(req, res, next) {
  console.log(req.method, req.url);
  next();   // sıradakine geç — çağırmazsan istek asılı kalır!
}

app.use(logger);              // tüm isteklere uygula
app.use(express.json());   // hazır middleware: JSON body → req.body
```

- `req`, `res`: route'takiyle **aynı** nesneler — üzerlerinde değişiklik yapılabilir
- `next`: "işim bitti, sıradakine geç" fonksiyonu

# Bir middleware 4 şeyden birini yapar

| Ne yapar? | Nasıl? | Sonuç |
|---|---|---|
| **Devam ettirir** | `next()` | Sıradaki middleware/route çalışır |
| **Zinciri keser** | `res.status(401).json(...)` | Yanıt gider, sonrakiler **çalışmaz** |
| **Hataya atlar** | `next(err)` veya `throw` | Normal middleware'ler atlanır, **hata middleware'ine** gidilir |
| **Bilgi ekler** | `req.x = ...; next()` | Sonraki herkes `req.x`'i görür |

⚠️ Hiçbirini yapmazsa (ne `next()` ne yanıt) istek **asılı kalır** → istemci timeout alır.

# Çalışma sırası

```
app.use((req, res, next) => { console.log('1. A');     next(); });
app.use((req, res, next) => { console.log('2. B');     next(); });

app.get('/', (req, res) => { console.log('3. route');  res.send('ok'); });

app.use((req, res, next) => { console.log('4. ???');   next(); });
```

`GET /` isteği için çıktı:

```
1. A
2. B
3. route
```

`4. ???` **hiç çalışmaz** — route yanıtı gönderdi, zincir bitti.

Ama `GET /olmayan-yol` için çalışır → 404 middleware'i bu yüzden **en sonda**.

# `req`'e bilgi eklemek

```
app.use((req, res, next) => {
  req.istekId = Math.random().toString(36).slice(2, 8);
  res.set('X-Istek-Id', req.istekId);
  next();
});

app.get('/api/istek', (req, res) => {
  res.json({ istekId: req.istekId });   // middleware'in eklediği bilgi
});
```

Bu kalıp çok yaygın:

- Auth middleware'i → `req.user` (Part 2)
- Logger → `req.istekId` (hata loglarını eşleştirmek için)
- Dil algılama → `req.dil`

# Middleware'i nereye bağlarız?

| Kapsam | Kod | Ne zaman çalışır? |
|---|---|---|
| **Tüm uygulama** | `app.use(fn)` | Her istekte |
| **Yol öneki** | `app.use('/api', fn)` | `/api`, `/api/...` ile başlayan her istekte |
| **Tek route** | `app.get('/gizli', fn, handler)` | Sadece o route'ta |
| **Birden fazla** | `app.get('/x', [a, b], handler)` | Sırayla a, b, sonra handler |
| **Router** | `router.use(fn)` | O router'a bağlı route'larda (Part 2: mimari) |

# Route bazlı middleware

```
function apiAnahtariGerekli(req, res, next) {
  if (req.get('X-API-Key') !== 'gizli123') {
    return res.status(401).json({ hata: 'Geçersiz API anahtarı' });
  }
  next();
}

app.get('/api/gizli', apiAnahtariGerekli, (req, res) => {
  res.json({ mesaj: 'Hoş geldin!' });
});
```

Part 2'de gerçek kimlik doğrulama (JWT) tam olarak bu kalıpla yapılacak.

# Hazır middleware'ler

| Middleware | Kaynak | Ne yapar? |
|---|---|---|
| `express.json()` | Express içinde | JSON body → `req.body` |
| `express.urlencoded()` | Express içinde | HTML form verisi → `req.body` |
| `express.static('public')` | Express içinde | Klasördeki dosyaları (HTML, resim) sunar |
| `cors` | npm paketi | Başka domain'lerden gelen tarayıcı isteklerine izin verir |
| `helmet` | npm paketi | Güvenlik header'ları ekler |
| `morgan` | npm paketi | Hazır istek logger'ı |
| `express-rate-limit` | npm paketi | Aynı IP'den çok fazla isteği engeller |

Hepsi aynı `(req, res, next)` imzasını kullanır — sihir yok, bizim yazdıklarımızla aynı mantık.

# Hata yönetimi: `next(err)` ve `throw`

```
app.get('/api/sayi/:n', (req, res, next) => {
  const n = Number(req.params.n);
  if (Number.isNaN(n)) {
    const err = new Error('n bir sayı olmalı');
    err.status = 400;
    return next(err);                  // ← bilerek hata middleware'ine gönder
  }
  res.json({ n, karesi: n * n });
});

// Express v5: async route'ta throw → otomatik hata middleware'ine 🎉
app.get('/api/patla', async (req, res) => {
  throw new Error('Bir şeyler ters gitti!');
});
```

v4'te async hatalar yakalanmaz, sunucu çökebilirdi → `try/catch` + `next(err)` gerekirdi.

# 404 ve hata middleware'i

```
// ... tüm route'lar yukarıda ...

// 404 — hiçbir route eşleşmezse buraya düşer
app.use((req, res) => {
  res.status(404).json({ hata: 'Bulunamadı' });
});

// Hata middleware'i — 4 parametre (err ilk sırada), EN SONDA
app.use((err, req, res, next) => {
  console.error(err);
  const status = err.status ?? 500;
  res.status(status).json({
    hata: status === 500 ? 'Sunucu hatası' : err.message,  // iç detayı sızdırma!
  });
});
```

Express hata middleware'ini **parametre sayısından** (4) tanır — `next`'i kullanmasan da yazmalısın.

# Middleware'de doğru sıralama

```
1. Logger              ← her şeyi görsün (hatalı istekleri de)
2. Güvenlik/CORS       ← erken reddet
3. Body parser         ← express.json()
4. Auth (gerekiyorsa)
5. Route'lar
6. 404
7. Hata middleware'i ← her zaman en son
```

📂 `npm run ornek:4` — logları izleyerek her isteğin zincirde nasıl ilerlediğini gösterin.

# 6. Canlı kodlama: Todo API

# REST: kaynak odaklı tasarım

URL'ler **isim** (kaynak), metotlar **fiil** (işlem):

| İşlem | Metot + Yol | Başarı | Hata |
|---|---|---|---|
| Listele | `GET /todos` | 200 | — |
| Getir | `GET /todos/:id` | 200 | 404 |
| Oluştur | `POST /todos` | 201 | 400 |
| Güncelle | `PATCH /todos/:id` | 200 | 400, 404 |
| Sil | `DELETE /todos/:id` | 204 | 404 |

❌ `POST /createTodo`, `GET /deleteTodo?id=3`

✅ `POST /todos`, `DELETE /todos/3`

# Hadi yazalım!

📂 `ornekler/05-todo-api/baslangic.js` → `npm run ornek:5`

```
let todos = [
  { id: 1, title: 'Node.js kur', done: true },
  { id: 2, title: 'Express öğren', done: false },
];
let nextId = 3;
```

- Veri şimdilik **bellekte** (sunucu yeniden başlarsa sıfırlanır)
- Test için: `istekler.http` (VS Code REST Client) veya `curl`

# Validation: kullanıcıya asla güvenme

```
app.post('/todos', (req, res) => {
  const { title } = req.body ?? {};

  if (typeof title !== 'string' || title.trim() === '') {
    return res.status(400).json({ error: 'title zorunlu' });
  }

  const todo = { id: nextId++, title: title.trim(), done: false };
  todos.push(todo);
  res.status(201).json(todo);
});
```

Frontend'de kontrol olsa bile **backend tekrar kontrol eder** — herkes curl ile istek atabilir.

# Sorun: sunucu kapanınca her şey gidiyor

```
POST /todos  "Süt al"   →  todos dizisi: [..., { id: 3, title: 'Süt al' }]
Ctrl+C, sunucu yeniden başlar
GET /todos              →  [{ id: 1 }, { id: 2 }]   😱  "Süt al" nerede?
```

- Değişkenler **RAM'de** yaşar → süreç bitince silinir
- Kalıcılık için veriyi **diske** yazmamız gerekir
- En basit yol: veriyi bir **JSON dosyasına** yazmak

Node'un dahili `node:fs/promises` modülü dosya okuma/yazmayı **async** olarak yapar → beklerken sunucu diğer isteklere bakmaya devam eder.

# Mini JSON veritabanı: `db.js`

```
import { readFile, writeFile, mkdir } from 'node:fs/promises';

const KLASOR = new URL('./data/', import.meta.url);
const DOSYA = new URL('todos.json', KLASOR);

export async function readTodos() {
  try {
    return JSON.parse(await readFile(DOSYA, 'utf8'));  // dosya → string → dizi
  } catch (err) {
    if (err.code === 'ENOENT') return [];   // dosya yok → ilk çalıştırma
    throw err;
  }
}

export async function writeTodos(todos) {
  await mkdir(KLASOR, { recursive: true });                // klasör yoksa oluştur
  await writeFile(DOSYA, JSON.stringify(todos, null, 2)); // dizi → string → dosya
}
```

# Route'larda kullanmak: oku → değiştir → yaz

```
import { readTodos, writeTodos } from './db.js';

app.post('/todos', async (req, res) => {
  // ... validation ...
  const todos = await readTodos();                    // 1) oku
  const todo = { id: nextId(todos), title, done: false };
  todos.push(todo);                                   // 2) değiştir
  await writeTodos(todos);                            // 3) geri yaz
  res.status(201).json(todo);
});
```

**Demo** 📂 `npm run ornek:6`: görev ekle → `Ctrl+C` → tekrar başlat → `GET /todos` → görevler **hâlâ orada** 🎉

Sonra `data/todos.json` dosyasını açıp verinin düz metin olarak durduğunu gösterin.

# JSON dosyası neden gerçek veritabanı değil?

| Sorun | Ne olur? | Gerçek veritabanı |
|---|---|---|
| **Her istekte tüm dosya** okunur/yazılır | 1 milyon görevde çok yavaş | Sadece gereken satırı okur/yazar |
| **Aynı anda iki istek** yazarsa | Biri diğerinin değişikliğini ezer | Transaction ve kilitleme |
| Yazarken sunucu çökerse | Dosya yarım kalır, bozulur | Çökmeye dayanıklı yazma |
| Sorgulama ("yapılmamış ve bu hafta") | Hepsini oku, JS ile filtrele | `WHERE` ile veritabanı yapar |
| Birden fazla sunucu | Her biri kendi dosyasını görür | Tek merkezi veritabanı |

Ama **fikir aynı:** backend veriyi bir yerde **kalıcı** saklar.
Part 2'de bu `db.js`'in yerini gerçek bir veritabanı alacak — route'lar neredeyse hiç değişmeyecek.

# Özet

- **HTTP:** metot + yol + header + body → durum kodu + body
- **Node.js:** tek thread, bloklamayan I/O, async/await
- **Express:** routing, `req` / `res`, params, query
- **Middleware:** `(req, res, next)` zinciri, sıra önemli
- **REST:** kaynak odaklı URL'ler, doğru durum kodları
- **Validation:** gelen veriye güvenme
- **Kalıcılık:** RAM geçici, disk kalıcı — `fs/promises` ile JSON dosyası

# 7. Sırada ne var?

## Part 2 kavramlarına kısa bakış

# Veritabanı: neden?

Şu an verimiz bir dizide → sunucu yeniden başlayınca **her şey siliniyor**.

| | SQL (ilişkisel) | NoSQL (doküman) |
|---|---|---|
| Örnekler | PostgreSQL, MySQL, SQLite | MongoDB, Firestore |
| Yapı | Tablolar, satırlar, sütunlar | JSON benzeri dokümanlar |
| Şema | Sabit, önceden tanımlı | Esnek |
| İlişkiler | Güçlü (`JOIN`, yabancı anahtar) | Zayıf, genelde iç içe veri |
| Ne zaman? | Çoğu iş uygulaması | Esnek/değişken yapıdaki veri |

**İlişki örneği:** bir *kullanıcının* birçok *görevi* olur → `todos` tablosunda `user_id` sütunu.

# ORM: kod ile veritabanı arasında köprü

```
JavaScript kodu  ──►  ORM / Query Builder  ──►  SQL  ──►  Veritabanı
(nesneler)          (Prisma, Drizzle, Knex)              (tablolar)
```

- SQL'i elle yazmak yerine JavaScript fonksiyonlarıyla sorgu
- Tablo yapısını kodda tanımlarız → **migration** ile veritabanına uygulanır
- **Migration:** veritabanı şemasının versiyon kontrolü ("v3: todos tablosuna `due_date` eklendi")
- ⚠️ **SQL injection:** kullanıcı girdisini SQL metnine asla elle birleştirme — ORM'ler bunu bizim için önler

# Kimlik doğrulama vs yetkilendirme

| | **Authentication (kimlik doğrulama)** | **Authorization (yetkilendirme)** |
|---|---|---|
| Soru | **Sen kimsin?** | **Bunu yapmaya iznin var mı?** |
| Örnek | E-posta + şifre ile giriş | Sadece yazarı kendi görevini silebilir |
| Başarısızsa | `401 Unauthorized` | `403 Forbidden` |
| Ne zaman? | Önce | Sonra |

Havalimanı benzetmesi: **pasaport kontrolü** kimliği doğrular, **biniş kartı** hangi uçağa binebileceğini belirler.

# Şifreler nasıl saklanır?

**Asla düz metin olarak değil!** Veritabanı sızarsa tüm şifreler ifşa olur.

- **Hash:** tek yönlü fonksiyon. `"gizli123"` → `"$2b$10$N9qo8u..."`, geri döndürülemez
- Girişte: gelen şifreyi tekrar hashle → kayıtlı hash ile karşılaştır
- **Salt:** her kullanıcıya rastgele ek → aynı şifreler farklı hash üretir
- **Yavaş algoritmalar** kullanılır: `bcrypt`, `argon2` (deneme-yanılma saldırılarını zorlaştırmak için)
- ❌ MD5, SHA-1 gibi hızlı hash'ler şifre için **uygun değil**

**Hash ≠ şifreleme (encryption):** şifrelenen veri anahtarla geri açılır, hash açılmaz.

# Giriş yaptıktan sonra: Session vs JWT

HTTP durumsuz → her istekte "ben giriş yapmıştım" diye kanıtlamak gerekir.

| | Session | JWT (token) |
|---|---|---|
| Nasıl? | Sunucu oturumu saklar, istemciye **cookie** ile ID verir | Sunucu **imzalı bir token** verir, istemci her istekte gönderir |
| Durum nerede? | Sunucuda (DB/Redis) | Token'ın içinde |
| Çıkış yapma | Kolay: oturumu sil | Zor: token süresi dolana kadar geçerli |
| Uygun olduğu yer | Klasik web siteleri | API'ler, mobil uygulamalar |

Akış: `POST /login` → token al → sonraki isteklerde `Authorization: Bearer <token>` → **auth middleware'i** doğrular → `req.user`

# Katmanlı mimari

Tek dosyada 50 route → okunamaz, test edilemez. Sorumlulukları ayırırız:

```
İstek ─►  Route       "Hangi URL hangi fonksiyona gider?"
             │
             ▼
          Controller  "HTTP'yi anla: req'ten veriyi al, res ile yanıt ver"
             │
             ▼
          Service     "İş kuralları: görev en fazla 10 tane olabilir"
             │
             ▼
          Repository  "Veritabanı ile konuş"
```

- Her katman sadece bir altındakini bilir
- Service HTTP bilmez → aynı kurallar CLI'dan, kuyruktan da çağrılabilir
- Veritabanı değişirse sadece Repository değişir

# Validation kütüphaneleri

Bugün `if (typeof title !== 'string' ...)` ile elle kontrol ettik. 20 alanlı bir form için?

- **Şema** tanımlarız: "title: zorunlu metin, 1-200 karakter; done: boolean, opsiyonel"
- Kütüphane gelen veriyi şemaya göre kontrol eder, **tüm hataları** bir kerede listeler
- Örnekler: **Zod**, Joi, Yup (Fastify'da JSON Schema dahili)
- Bonus: şemadan otomatik **dokümantasyon** (OpenAPI/Swagger) ve TypeScript tipleri

Kural aynı: **gelen hiçbir veriye güvenme** — body, query, params, header.

# Yapılandırma ve CORS

**Ortam değişkenleri (environment variables)**

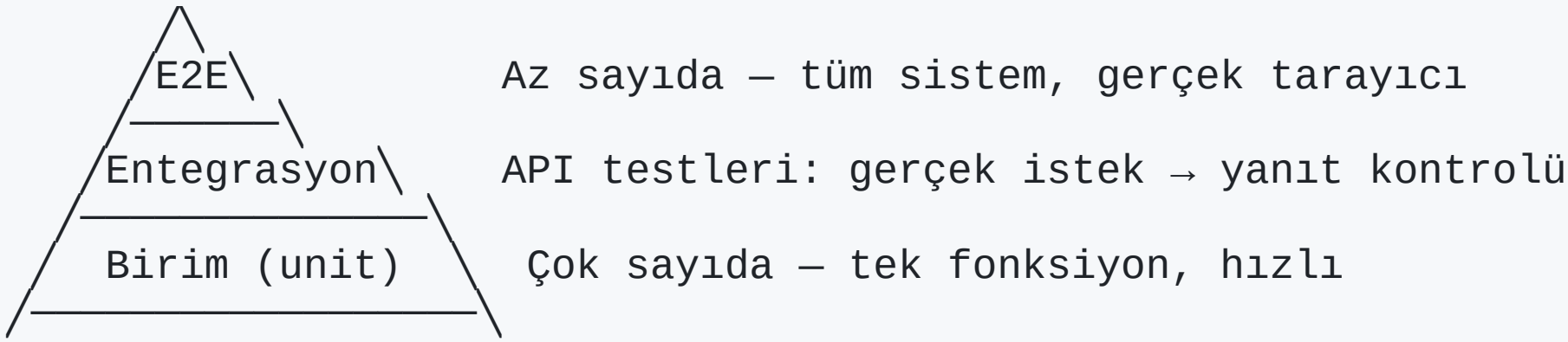
- Port, DB adresi, JWT anahtarı gibi değerler **koda yazılmaz**
- `.env` dosyasında tutulur, **asla git'e eklenmez**
- Aynı kod farklı ortamlarda farklı ayarla çalışır: geliştirme / test / üretim

**CORS (Cross-Origin Resource Sharing)**

- Tarayıcı, `site-a.com`'daki sayfanın `api-b.com`'a istek atmasını **varsayılan olarak engeller**
- Sunucu header'larla "bu domain'lere izin veriyorum" der
- `curl` ve Postman'de sorun yok — bu bir **tarayıcı** güvenlik kuralı

# Test

Kodu her değiştirdiğinde tüm uçları elle curl'lemek mi? ❌




- **Birim:** `validateTitle('')` → `false` mu?
- **Entegrasyon:** `POST /todos` boş title ile → `400` dönüyor mu?
- Araçlar: Node'un dahili `node:test`, Vitest, Jest, Supertest

# Part 2 özeti

- 🗄️ **Veritabanı:** SQL, ilişkiler, ORM, migration
- 🔐 **Kimlik:** hash'lenmiş şifreler, JWT, auth middleware'i, 401 vs 403
- 🏗️ **Mimari:** route → controller → service → repository
- ✅ **Validation:** şema tabanlı (Zod)
- ⚙️ **Yapılandırma:** ortam değişkenleri, CORS
- 🧪 **Test:** birim ve API testleri

Todo API'mizi bu adımlarla **gerçek bir uygulamaya** dönüştüreceğiz.

# Sorular?

Kodlar: `ornekler/` · İstekler: `istekler.http`